



POLITECNICO
MILANO 1863



IoT Challenge #3

Node-Red

+

LoRaWAN

Challenge 3: Node-Red and LoRaWAN

**Part 1: Challenge
Node-Red**

**Part 2: Exercise
LoRaWAN**

PART 1 - Challenge

Part 1: Challenge – Node Red

- Download the challenge3.csv file from WeBeep
- Process the CSV file in Node-Red

```

Packet Number,Device Name Source,Device Type Source,Source Address,Destination Address,Device Name Destination,Device Type Destination,Source Address ZigBee,
0,Coordinator,Coordinator,0x0000,0xffff,Broadcast,Broadcast,0x0000,Coordinator,Coordinator,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifi
1,Power Plug 1,Socket,0x3d95,0x0000,Coordinator,Coordinator,0x3d95,Power Plug 1,Socket,0x0000,Coordinator,Coordinator,0,,Report Attributes (0x0a),{"Layer ZBEE
2,Coordinator,Coordinator,0x0000,0xec7f,Ledvance Bulb,Bulb,0x0000,Coordinator,Coordinator,0x3d95,Power Plug 1,Socket,0,,Default Response (0x0b),{"Layer ZBEE
3,Ledvance Bulb,Bulb,0xec7f,0x3d95,Power Plug 1,Socket,0x0000,Coordinator,Coordinator,0x3d95,Power Plug 1,Socket,0,,Default Response (0x0b),{"Layer ZBEE_ZCU
4,Ledvance Z3 Plug,Socket,0x4e11,0xffff,Broadcast,Broadcast,0x4e11,Ledvance Z3 Plug,Socket,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifi
5,Ledvance Bulb,Bulb,0xec7f,0xffff,Broadcast,Broadcast,0xec7f,Ledvance Bulb,Bulb,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifier": 'L3
6,Philips Lamp 1,Bulb,0x1cd8,0xffff,Broadcast,Broadcast,0x1cd8,Philips Lamp 1,Bulb,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifier": '
7,Philips Motion,Motion,0x059b,0x1cd8,Philips Lamp 1,Bulb,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,1,device,Report Attributes (0x0a),{"La
8,Philips Lamp 1,Bulb,0x1cd8,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,0,,Route Record (0x05),{"Layer ZBEE
9,Philips Lamp 1,Bulb,0x1cd8,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,1,device,Report Attributes (0x0a),{"
10,Philips Lamp 1,Bulb,0x1cd8,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,1,device,Report Attributes (0x0a),{"
11,Coordinator,Coordinator,0x0000,0x1cd8,Philips Lamp 1,Bulb,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0,,Default Response (0x0b),{"Layer
12,Smart Socket,Socket,0xe1a6,0xffff,Broadcast,Broadcast,0xe1a6,Smart Socket,Socket,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifier":
13,Philips Lamp 1,Bulb,0x1cd8,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0,,Default Response (0x0b),{"Layer ZBEE
14,Philips Lamp 1,Bulb,0x1cd8,0x0000,Coordinator,Coordinator,0x059b,Philips Motion,Motion,0x0000,Coordinator,Coordinator,0,,Route Record (0x05),{"Layer ZBEE
15,Moes Bulb,Bulb,0x1e15,0xffff,Broadcast,Broadcast,0x1e15,Moes Bulb,Bulb,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifier": 'Link Stat
16,Power Plug 2,Socket,0xa706,0xffff,Broadcast,Broadcast,0xa706,Power Plug 2,Socket,0xfffc,Broadcast,Broadcast,0,,Link Status (0x08),{"Command Identifier":

```



Packet Number!

Challenge: What to do? (1) 1 point

- Create a flow to periodically publish MQTT messages to the local mosquitto broker (**localhost, port 1884**), to the topic ***challenge3/id_generator***
(be sure to start the mosquitto broker locally with the correct port)
- Messages should be sent with a rate of **1 message every 1 second**.
- Each message should contain in the payload a JSON-formatted string with a **random** number (**id**) between **0 and 30000**, and the time at which the message is generated (**UNIX timestamp**)

Message payload example: {"id":7123, "timestamp":1710930219}

When sending the message, also save its fields in a CSV (***id_log.csv***) in the form:
No.,ID,TIMESTAMP where No. is the incremental row number

Include this CSV in your delivery

What to do? (2) 2 points

In another branch of the flow (same flow):

- **Subscribe to the topic *challenge3/id_generator* in the local broker (localhost, port 1884)**
- After receiving a message from the subscription, take the ID and compute the **remainder after division by 5218** to get **N**:
N = ID modulo 5218
- For every message received, process the **challenge3.csv** file and take the message whose packet number is equal to the received identifier **N**

e.g. id= 7123

N = 7123 modulo 5218 = 1905

What to do? (2) 2 points

- If the message with Packet No. = N in the file contains (under “Command String” column) a ZBEE_ZCL Layer, perform an MQTT Publish to the topic named after the value in “Device Name ZigBee Source” corresponding column

The message you publish should use the following payload:

```
{"timestamp": CURRENT_TIMESTAMP,"id": SUB_ID,"wpan.src": "SRC_ADDR","wpan.dst":  
"DST_ADDR","zbee.src": "ZB_SRC_ADDR","zbee.dst": "ZB_DST_ADDR","topic":  
"ZigBee/ZB_SRC_DEV_NAME","payload": CMD_STR}
```

- All the addresses must be in hexadecimal format e.g. 0x123ab

What to do? (2) 2 points

Where:

- CURRENT_TIMESTAMP **Current UNIX time** at the moment the publish message is sent
- SUB_ID Message **ID** received from the subscription i.e. **7123**
- SRC_ADDR: Source Address field from the CSV row with packet number **N**
- DST_ADDR: Destination Address field from the CSV row with packet number **N**
- ZB_SRC_ADDR: Source Address ZigBee field from the CSVrow with packet number **N**
- ZB_DST_ADDR: Destination Address ZigBee field from the CSV row with packet number **N**
- ZB_SRC_DEV_NAME: Device Name ZigBee Source field from the CSV row with packet number **N**
- CMD_STR: Command String field from the CSV row with packet number **N**
- Limit the messages published in this step to a rate of **ten messages per minute**
(**Use the rate limiter node**)

What to do? (2) 2 points

- In addition, after sending the publish message, if the Command String contains an RMS Current or an RMS Voltage attribute or an Active Power attribute, save selected elements in a CSV (filtered_elems.csv) as follows:

Given the following command string:

```
{'Layer ZBEE_ZCL': '', 'Sequence Number': '99', 'Command': 'Read Attributes Response (0x01)', 'Attribute': ['Active Power (0x050b)', 'RMS Current (0x0508)', 'RMS Voltage (0x0505)'], 'Status': ['Success (0x00)', 'Success (0x00)', 'Success (0x00)'], 'Data Type': ['16-Bit Signed Integer (0x29)', '16-Bit Unsigned Integer (0x21)', '16-Bit Unsigned Integer (0x21)'], 'Int16': '0', 'Uint16': ['0 (0x0000)', '223 (0x00df)']}
```

The extracted elements must be matched by position in their respective lists. This means that each attribute is associated with the status, data type, and value appearing in the same ordered position in the corresponding fields.

In the example:

Attribute[0] matches Status[0], Data Type[0], and the first corresponding value

Attribute[1] matches Status[1], Data Type[1], and the second corresponding value

Attribute[2] matches Status[2], Data Type[2], and the third corresponding value

What to do? (2) 2 points

10

filtered_elems.csv format:

No., Timestamp, Sequence Number, Attribute, Status, Data Type, Data Value

1,152952012,99,Active Power,Success,16-Bit Signed Integer,0

2,152952012,99,RMS Current,Success,16-Bit Unsigned Integer,0

3,152952012,99,RMS Voltage,Success,16-Bit Unsigned Integer,223

**where No. is row number
(incremental)**

- Finally plot the RMS Current and RMS Voltage associated value (e.g. 0 for RMS Current and 223 for RMS Voltage) in two Node-Red charts one for RMS Voltage and one for RMS Current.

Include this CSV in your delivery

What to do? (3) 2 points

- If the message with **Packet No. = N** instead contains a Link Status message (**Packet type = Link Status (0x08)**), save the outgoing cost for each link associated with that ZigBee source address.
- If a later Link Status packet arrives from the same source, update the stored cost for each destination.
- At the end of the execution (which will be described later) save the cost for each link in a “**outgoing_cost.csv**” CSV file with the following fields:

No.,Source,Destination,Cost

Where:

- **Source** is the ZigBee source address field
- The **Destination** is the address ...
- **Cost** is corresponding outgoing cost

Include this CSV in your delivery

What to do? (3) 2 points

- Find the smallest source address in outgoing_cost.csv, treating the addresses as hexadecimal numbers.
- For that source address, sort its destination addresses in ascending hexadecimal order.
- Then send the outgoing cost for each destination address to your ThingSpeak channel using the HTTP API. For each message, put the outgoing cost in field1. Send the messages in destination-address order, and respect a rate limit of one message every 20 seconds.

Include the channel link in the report and in the form, **make it public!!**

What to do? (3) 2 points

- In all the other cases (Packet No = **N** not containing an ZBEE_ZCL Layer or a Link Status) Ignore the message!

Program your flow to stop working after **receiving exactly 200 ID** messages from the subscription:

do not process more than 200 ID messages

Ignored messages still count toward the 200-message limit.

Recap on CSV File Formatting

The first row should include the header.

Report the various fields separated by a commas.

- **id_log.csv** format:

No.,ID,TIMESTAMP

e.g. *1,7123,152952003*

- **filtered_elems.csv** format:

No.,Timestamp,Sequence Number, Attribute,Status,Data Type,Data Value

e.g. *1,152952012,99,Active Power,Success,16-Bit Signed Integer,0*

- **outgoing_cost.csv** format:

No.,Source,Destination,Cost

e.g. *1,0x0000,0x0112,7*

PART 2 - Exercise

Exercise Questions (EQ1-EQ2)

A LoRaWAN network in Europe operates at carrier frequency **868 MHz** with **bandwidth 125 kHz**. The network is composed of **one gateway** and **40 sensor nodes**. Each sensor node transmits packets with **payload size 20 bytes** according to a Poisson process with intensity **$\lambda=2$ packets/minute**.

EQ1) Determine the largest LoRa spreading factor (SF) that guarantees a packet success rate of at least 75%. Assume that all nodes use the same SF and adopt a simplified model where packet transmissions are uncoordinated and may overlap in time. To compute the airtime of a packet, use the TTN airtime calculator: <https://www.thethingsnetwork.org/airtime-calculator>

EQ2) After deploying the network in the field, the measured packet success rate is significantly lower than expected. Moreover, performance is not uniform across nodes: some nodes achieve relatively good delivery rates, while others experience very poor performance.

Which of the following actions is the most appropriate? Briefly justify your answer.

1. Decrease the number of nodes
2. Move the nodes closer to the gateway
3. Increase the spreading factor

Challenge deliverables

PART 1 – Challenge:

- A PDF report “Challenge.pdf” containing the explanation of the Node-Red nodes. Include an image of the Node-Red flow, explain the meaning of each node. Report also the pictures of the obtained node-red Chart!
- Node-Red flow export as JSON: *nodered.txt*
- CSV files produced: *id_log.csv*, *filtered_elems.csv*, and *outgoing_cost.csv*
- Thingspeak channel ID (make it public)

PART 2 – Exercise:

- A PDF “Exercise.pdf” containing the answers to EQ1-EQ2